
Sistemas Distribuidos

Práctica individual (C/sockets)

Red P2P RING

NOTA: No sustituye al enunciado

Introducción

- Plazos: ordinaria: 22 de junio; extraordinaria: 14 de julio
- Desarrollar en máquina con **Linux** y entorno C
 - Máquina personal, escritorio virtual Linux de la UPM o triqui.fi.upm.es
 - En cualquier caso, debe entregarse en triqui
- Ciclo de vida de la práctica:
 - Descarga de material de apoyo de página web de asignatura
 - Instalación material de apoyo
 - Desarrollo de (parte de) la funcionalidad pedida
 - Entrega de la práctica (solo desde triqui): **entrega.sd ring.2026**
 - Corrección automática (0, 12, 15, 18 y 21 horas)
 - Se activará en breve
 - Resultado de corrección → correo cuenta UPM del alumno
 - Número de entregas ilimitado
 - Última entrega se considera la versión definitiva

Objetivo

- Herramienta P2P para compartir contenidos
- Esquema P2P estructurado
- Topología en anillo
 - Como *Chord*, pero solo con búsqueda lineales sin *fingers*
- Uso de TCP: sockets *streams*
 - Para simplificar, no se mantienen conexiones persistentes
 - Una conexión por operación
- Operación *multithread* con creación dinámica de *threads*
 - 1 *thread* por petición/conexión
- Objetivo: minimizar copias entre zonas de memoria (*zerocopy*)
 - Recomendable para todas las transmisiones de la práctica
 - **Obligatorio** para la transmisión de los ficheros (uso *sendfile* y *mmap*)
- Se asume plataforma con formato de datos heterogéneo
 - *Big endian* vs. *Little endian*

Plataformas de desarrollo

- Equipos del centro de cálculo
 - triqui1.fi.upm.es triqui2.fi.upm.es triqui3.fi.upm.es triqui4.fi.upm.es
 - Pueden usarse para el desarrollo
 - Se debe entregar en cualquiera de estos equipos
- Escritorio virtual UPM (<https://escritorio.upm.es/uds/page/services>)
- Máquina personal con Linux nativo o alojado en m. virtual

Pruebas en un solo equipo

- Uso de localhost

Pruebas usando varios equipos

- Uso de triquis (a continuación, vemos un ejemplo de ejecución)
- Uso de varias máquinas virtuales conectadas (p.e. VirtualBox)
- Uso de Docker: cada contenedor representa un equipo
`docker run --rm -it -v $PWD:/RING ubuntu:24.04`

Descarga y compilación en triqui

descarga el material de apoyo

```
ssoo@triqui1:~$ wget
```

```
http://laurel.datsi.fi.upm.es/_media/docencia/asignaturas/sd/ring-2026.tgz
```

lo descomprime

```
ssoo@triqui1:~$ tar xvfz ring-2026.tgz
```

```
ssoo@triqui1:~$ cd DATSI/SSDD/ring.2026/src/
```

programamos la aplicación y la compilamos

```
ssoo@triqui1:src$ make
```

```
cc -Wall -g -linclude -c -o main.o main.c
```

```
cc -Wall -g -linclude -c -o ring_cln.o ring_cln.c
```

```
cc -Wall -g -linclude -c -o ring_srv.o ring_srv.c
```

```
cc -Wall -g -linclude -c -o common.o common.c
```

```
cc -o ring main.o ring_cln.o ring_srv.o common.o -lpthread
```

Ejecución en triquis (1/5)

```
ssoo@triqui1:src$ mkdir dir1 # directorio compartido  
ssoo@triqui1:src$ ./ring dir1 # solo un argumento: se crea la red  
Bienvenido a la red P2P RING
```

El equipo triqui1.fi.upm.es con IP 138.100.240.42 y puerto 43583 (PID 3115000) es el primer nodo de una nueva red

Seleccione operación (línea vacía para terminar; en menús internos para volver a menú principal)

I: obtiene Info de nodo local| P: getPid|S: Sucesor|R:sucesor Remoto|U: sUcesor de sucesor remoto|D: Download|L: Lookup fichero|G: Get fichero (lookup+download)

Ejecución en triquis (2/5)

```
ssoo@triqui2:src$ mkdir dir2 # directorio compartido  
ssoo@triqui2:src$ ./ring dir2 triqui1.fi.upm.es 43583 # se incorpora a la red  
Bienvenido a la red P2P RING
```

El equipo triqui2.fi.upm.es con IP 138.100.240.43 y puerto 40095 (PID 1488066) se incorpora a la red a través del equipo 138.100.240.42 puerto 43583

Seleccione operación (línea vacía para terminar; en menús internos para volver a menú principal)

I: obtiene Info de nodo local| P: getPid|S: Sucesor|R:sucesor Remoto|U: sUcesor de sucesor remoto|D: Download|L: Lookup fichero|G: Get fichero (lookup+download)

Ejecución en triquis (3/5)

```
ssoo@triqui3:src$ mkdir dir3
```

```
ssoo@triqui3:src$ ./ring dir3 triqui1.fi.upm.es 43583
```

Bienvenido a la red P2P RING

El equipo triqui3.fi.upm.es con IP 138.100.240.41 y puerto 36237 (PID 2359351) se incorpora a la red a través del equipo 138.100.240.42 puerto 43583

Seleccione operación (línea vacía para terminar; en menús internos para volver a menú principal)

I: obtiene Info de nodo local| P: getPid|S: Sucesor|R:sucesor Remoto|U: sUcesor de sucesor remoto|D: Download|L: Lookup fichero|G: Get fichero (lookup+download)

Ejecución en triquis (4/5)

```
ssoo@triqui4:src$ mkdir dir4  
ssoo@triqui4:src$ echo hola > dir4/f.txt # fichero a compartir  
ssoo@triqui4:src$ ./ring dir4 triqui2.fi.upm.es 40095  
Bienvenido a la red P2P RING
```

El equipo triqui4.fi.upm.es con IP 138.100.240.47 y puerto 35165 (PID 2311490) se incorpora a la red a través del equipo 138.100.240.43 puerto 40095

Seleccione operación (línea vacía para terminar; en menús internos para volver a menú principal)

I: obtiene Info de nodo local| P: getPid|S: Sucesor|R:sucesor Remoto|U: sUcesor de sucesor remoto|D: Download|L: Lookup fichero|G: Get fichero (lookup+download)

Ejecución en triquis (5/5)

- Volviendo a triqui1:

Seleccione operación (línea vacía para terminar; en menús internos para volver a menú principal)

I: obtiene Info de nodo local| P: getPid|S: Sucesor|R:sucesor Remoto|U: sUcesor de sucesor remoto|D: Download|L: Lookup fichero|G: Get fichero (lookup+download)

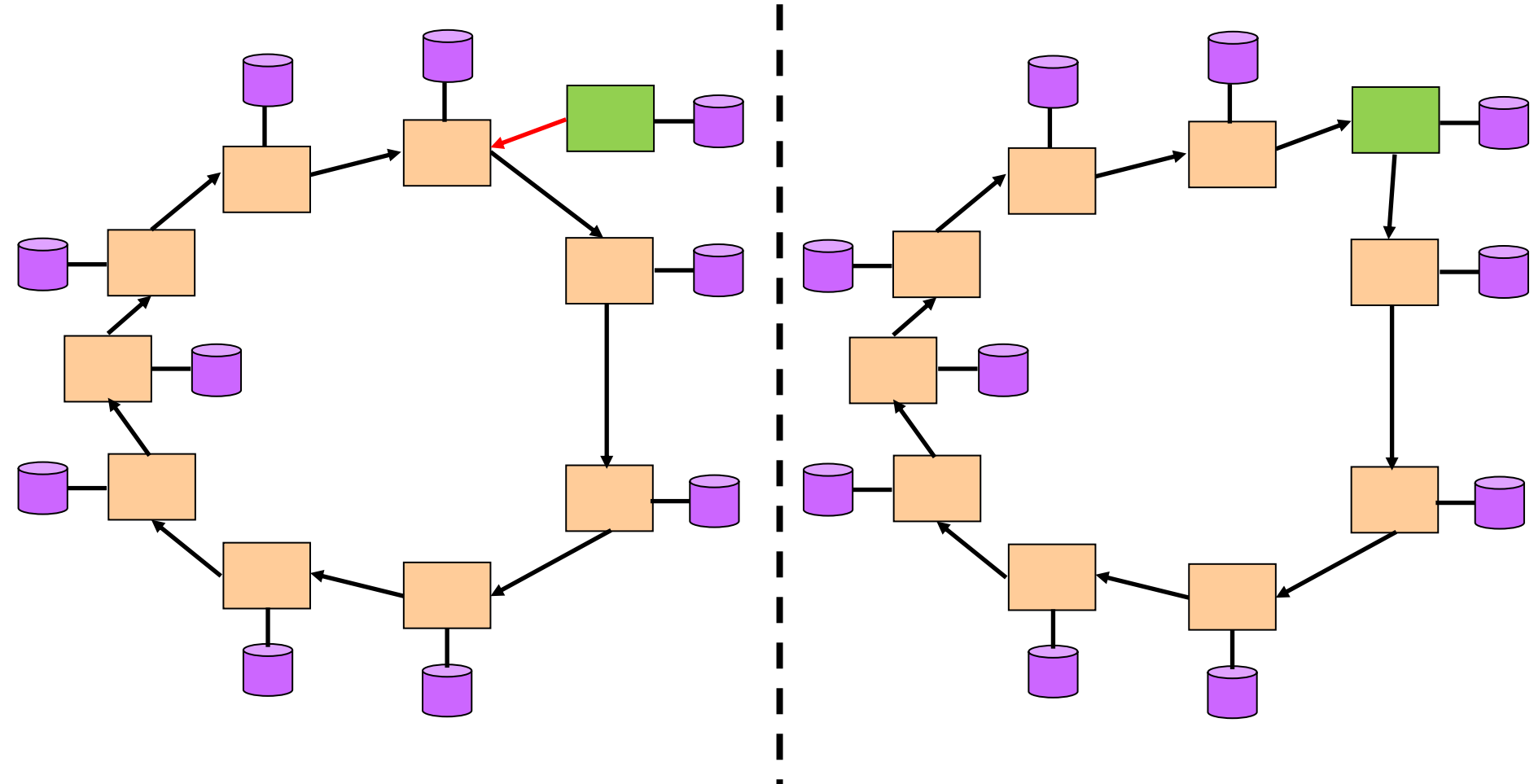
G

Introduzca el nombre del fichero: f.txt

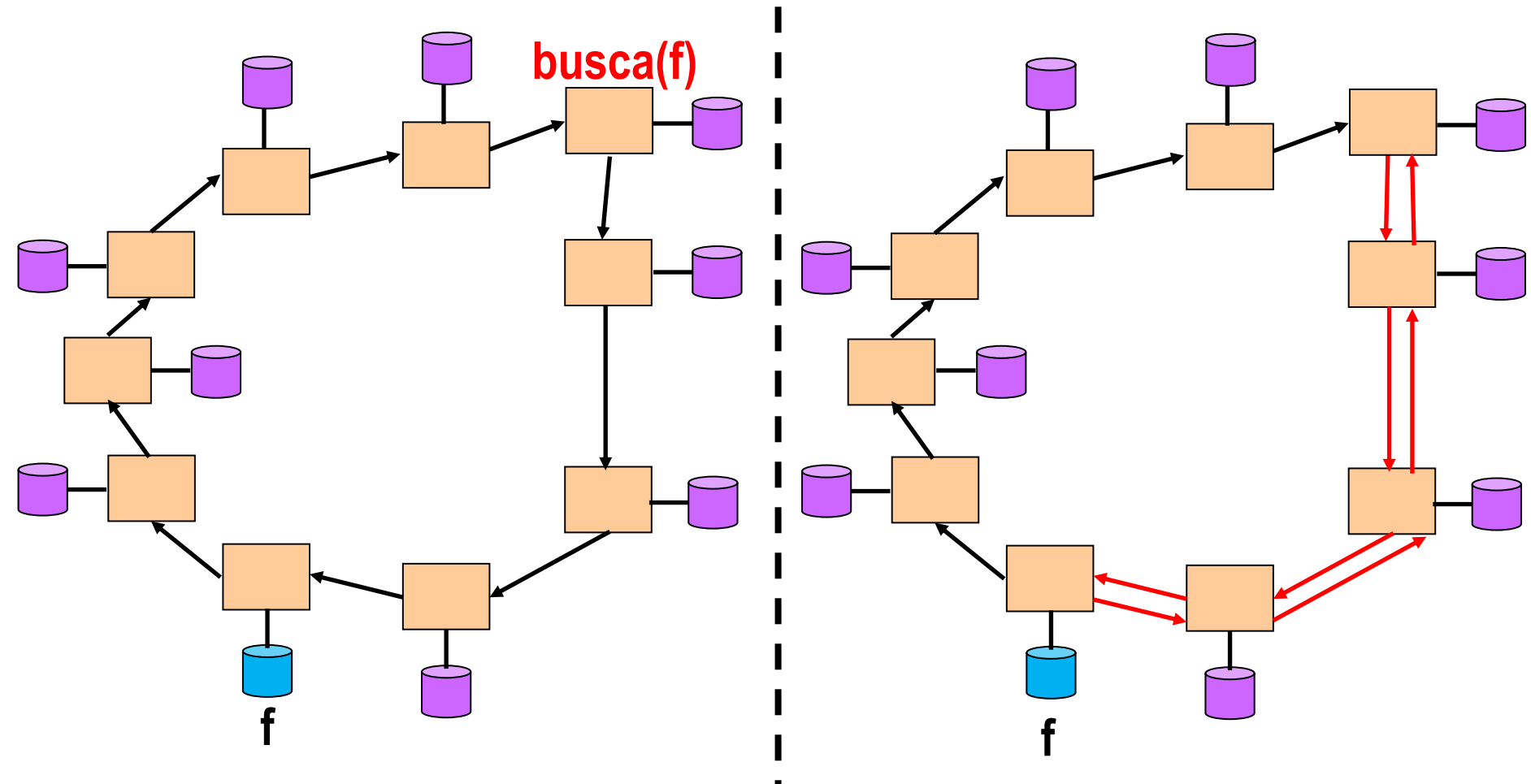
Introduzca el número máximo de nodos visitados: 4

ssoo@triqui1:src\$ cat dir1/f.txt # el fichero ha sido descargado
hola

Alta de un nodo



Búsqueda de un fichero



Arquitectura de la práctica

- P2P conlleva solo un rol: un nodo
- Un nodo tiene doble funcionalidad
- Parte cliente
 - Maneja interfaz de usuario
 - Implementa operaciones del usuario enviando un mensaje:
 - A la parte servidora del nodo especificado (p.e. en alta de nodo)
 - A la parte servidora del nodo sucesor (p.e. en *lookup*)
- Parte servidora
 - Gestiona las peticiones recibidas por la parte cliente de cualquier nodo
 - Petición puede causar envío de mensaje a parte servidora de sucesor
- Ambas partes deben estar activas simultáneamente
- En inicio de nodo
 - se crea un *thread* que ejecuta funcionalidad de la parte servidora

Organización del código

- Directorio ejemplo
- Directorio *src*
 - *Makefile*
 - *main*
 - *ring_cln*
 - *ring_srv*
 - *common*: funcionalidad común
- Directorio *src/include*
 - *ring.h* (solo lectura)
 - *common.h*

API de la parte cliente

```
int ring_init(const char *shared_dir, unsigned int local_ip, unsigned int
remote_ip, unsigned short remote_port, unsigned short *alloc_port);
int ring_self(unsigned int *ip, unsigned short *port);
int ring_remote_pid(unsigned int remote_ip, unsigned short remote_port);
int ring_successor(unsigned int *ip, unsigned short *port);
int ring_remote_successor(unsigned int remote_ip, unsigned short
remote_port, unsigned int *suc_ip, unsigned short *suc_port);
int ring_remote_successor_successor(unsigned int remote_ip, unsigned
short remote_port, unsigned int *suc_suc_ip, unsigned short *suc_suc_port);
int ring_download(unsigned int remote_ip, unsigned short remote_port, const
char *filename);
int ring_lookup(const char *filename, int hops, unsigned int *ip, unsigned short
*port);
int ring_get_file(const char *filename, int hops);
```


Diseño del servicio

- Hay que asignar ID a cada operación; alternativas:
 - carácter, entero, *string*...
- Parte cliente:
 - Envía operación con ID y parámetros a la parte servidora de
 - un determinado nodo o el sucesor
 - Recibe respuesta
- Parte servidora:
 - Por cada petición: una conexión y un *thread*
 - Recibe ID
 - Discrimina operación; alternativas
 - switch, if else if, vector de punteros a función...
 - Realiza acción
 - que puede conllevar el envío de mensaje a la parte servidora de sucesor
 - “recursividad en el nivel de red”
 - Envía respuesta a solicitante

Fases

- Implementación incremental
- Fase 1
 - Despliegue de la parte servidora
 - 1ª operación remota (PID de nodo remoto)
- Fase 2
 - Gestión de altas con mantenimiento del anillo
 - 2ª operación remota (sucesor de nodo remoto)
 - 1ª operación que usa el anillo (sucesor de sucesor de nodo remoto)
- Fase 3
 - Descarga directa del fichero del nodo correspondiente usando *sendfile*
- Fase 4
 - *lookup*: encuentra nodo que contiene el fichero
- Fase 5 (ya hecha): búsqueda (fase 4) + descarga (fase 3)

Sistemas Distribuidos

Repaso de sockets *stream* para la práctica

Envío de datos por socket stream

- Semántica similar a tubería
 - Por cada socket, SO usa un *buffer* de transmisión
- Envío (*send* o *write*): copia datos en ese *buffer*
 - Si hay sitio suficiente, retorna inmediatamente
 - Como solo copia, puede no devolver error aunque destino caído
 - Si no, bloqueo hasta que pueda copiar **todo**
 - Según vayan transmitiéndose datos va quedando espacio
 - Se desbloqueará cuando se haya copiado todo al *buffer*
- *send* normalmente devuelve valor = n° de bytes pedidos
- Pero puede devolver valor < en ciertas situaciones patológicas:
 - envío grande (bloqueante) y receptor cae o cierra socket sin leer todo
 - se trata de una condición de error
- Para la práctica puede usarse *send* o *write*
 - Devolviendo error si no se han transmitido todos los datos

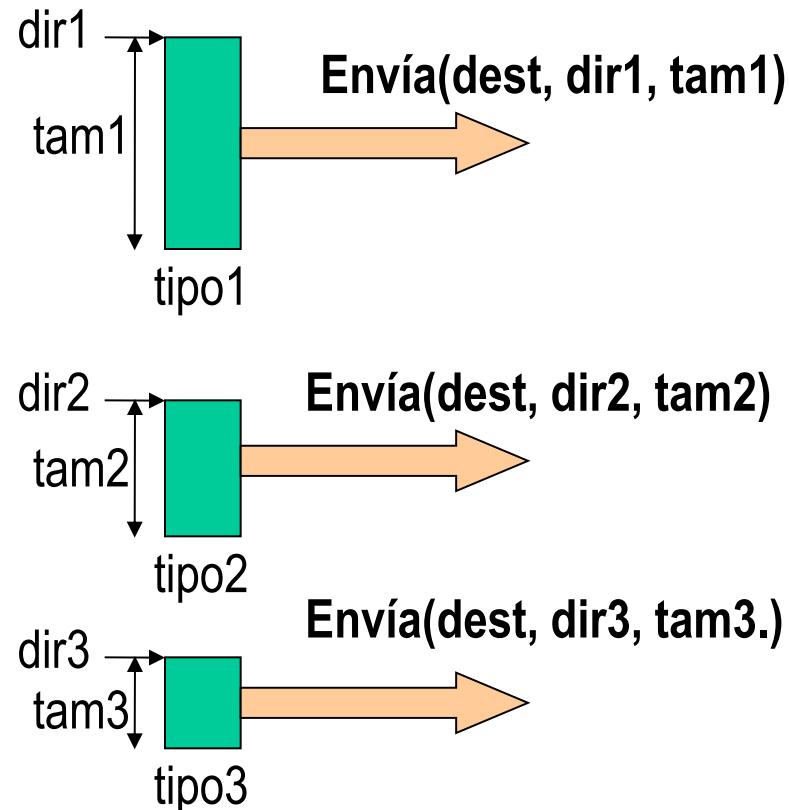
Recepción de datos por socket *stream*

- Puede devolver menos de los pedidos
- Devuelve 0 cuando el otro extremo cierra la conexión
- Muchas aplicaciones no encajan en el modelo *stream*
 - No pueden procesar un dato hasta que no lo tengan completo
- Varias alternativas para asegurar que se reciben N bytes
- Para la práctica puede usarse:
 - *flag MSG_WAITALL* en *recv*
`recv(s_conec, buf, tam, MSG_WAITALL)`

Zero-Copy

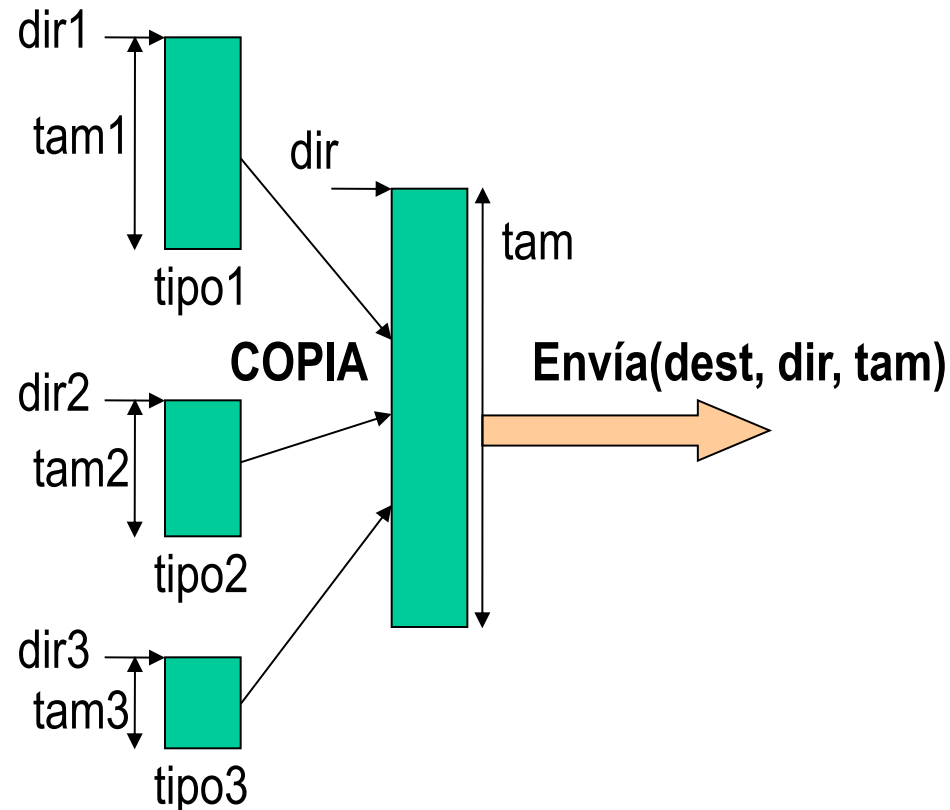
- Reducir al mínimo ($\approx$ a cero) copias entre zonas de memoria
 - SO y hardware intentan hacerlo
 - También lo debería intentar la aplicación
- *Escenario 1*: envío de N datos *dispersos* de emisor a receptor
 - N envíos: fragmentación de mensajes
 - Reserva de *buffer* y 1 envío: sobrecarga de copias
 - Solución 1: Función de envío de datos dispersos
 - Solución 2: Múltiples envíos pero indicando que hay más a continuación
- *Escenario 2*: envío de un fichero
 - Uso de operaciones convencionales de lectura y envío
 - Dos copias de memoria: de *buffer* de sistema a de usuario y viceversa
 - Uso de proyección de ficheros y envío
 - Una copia de memoria a *buffer* de sistema en envío
 - Uso de operaciones de transferencia directa entre descriptores
 - No requiere copias entres *buffers*; reduce nº llamadas al sistema

Datos dispersos: Envío múltiple



fragmentación de mensajes

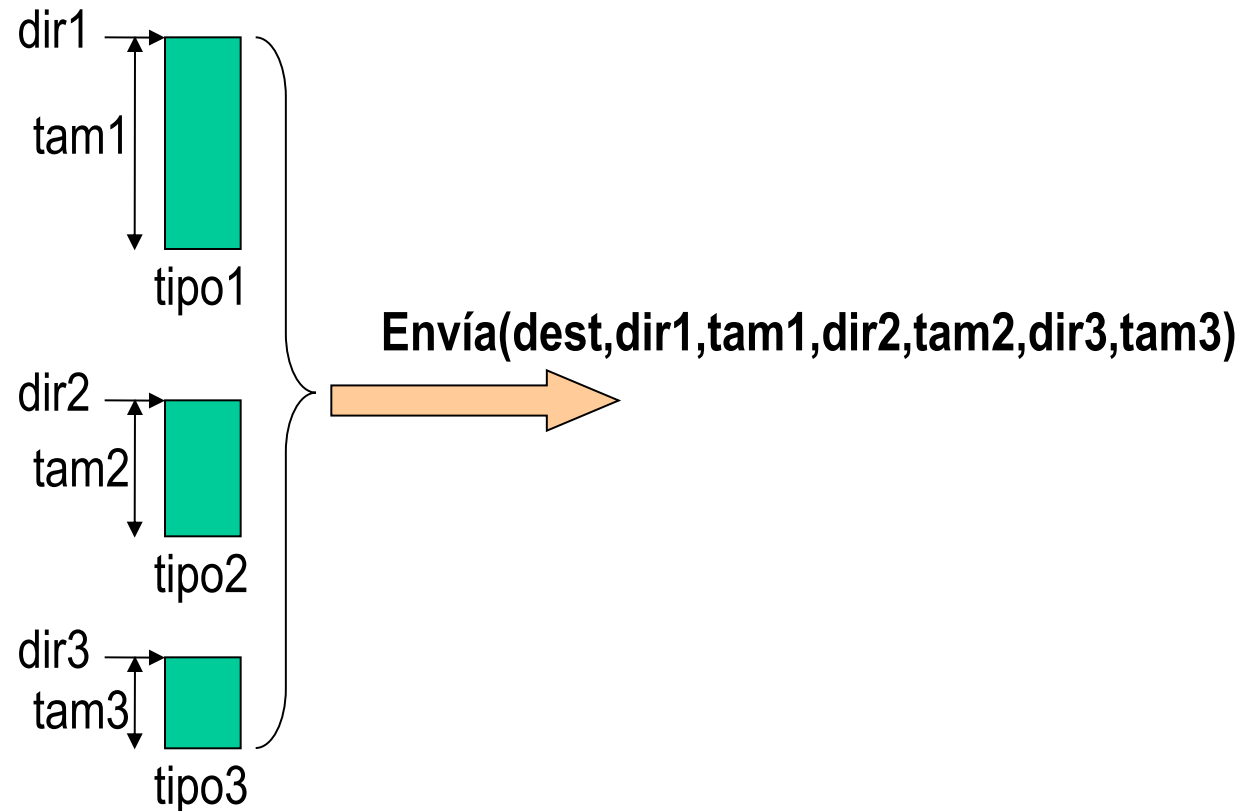
Datos dispersos: Envío con copia



sobrecarga por copias

Datos dispersos: Envío *gather* (solución 1)

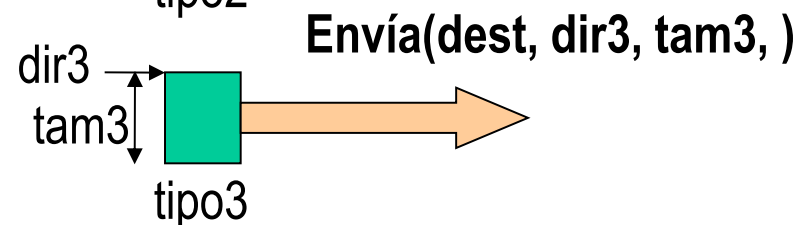
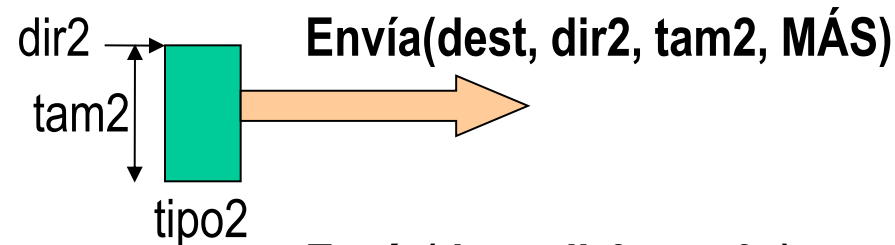
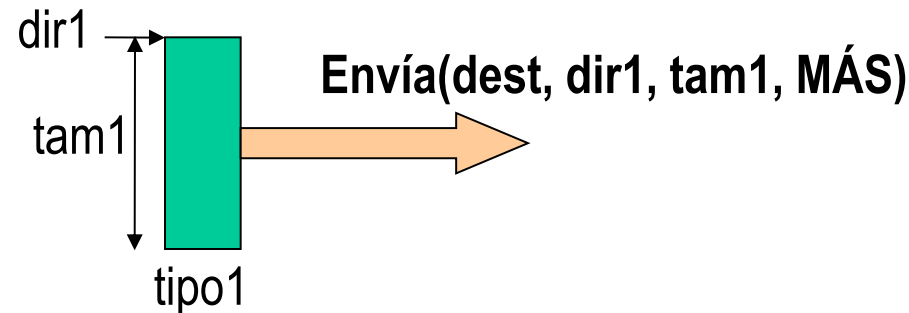
opción para la práctica



Uso de *writew* o *sendmsg*

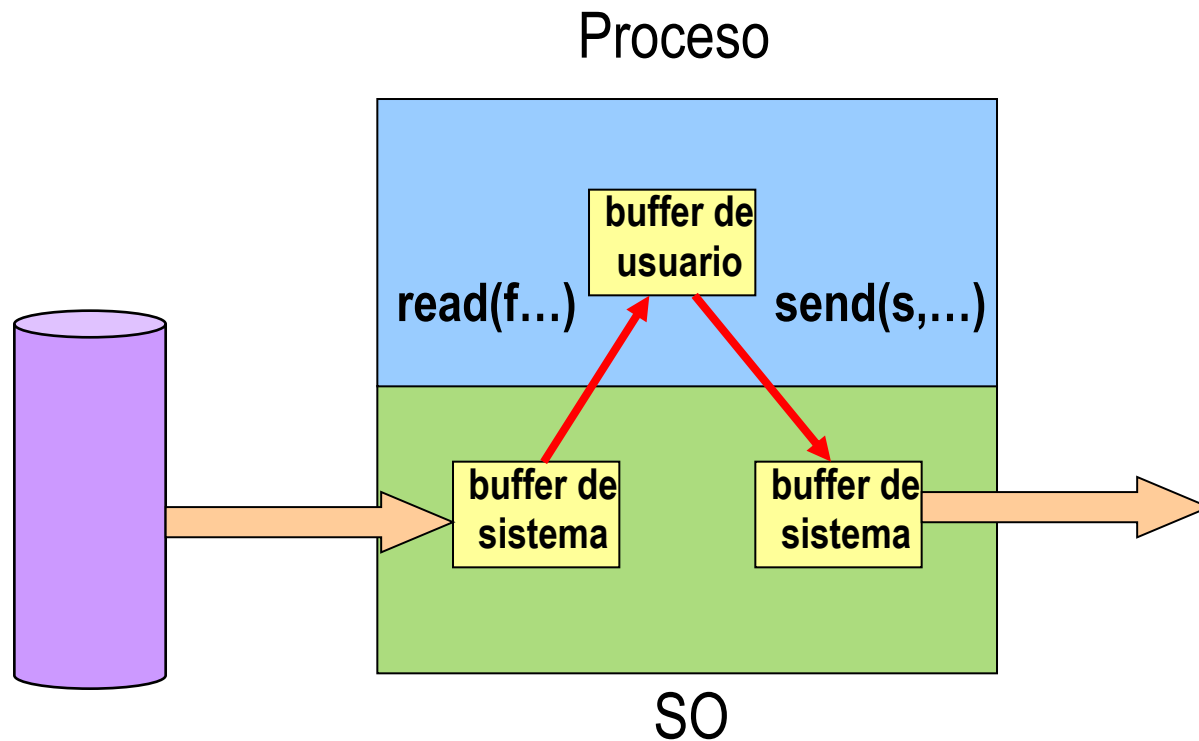
Datos dispersos: más mensajes (solución 2)

opción para la práctica



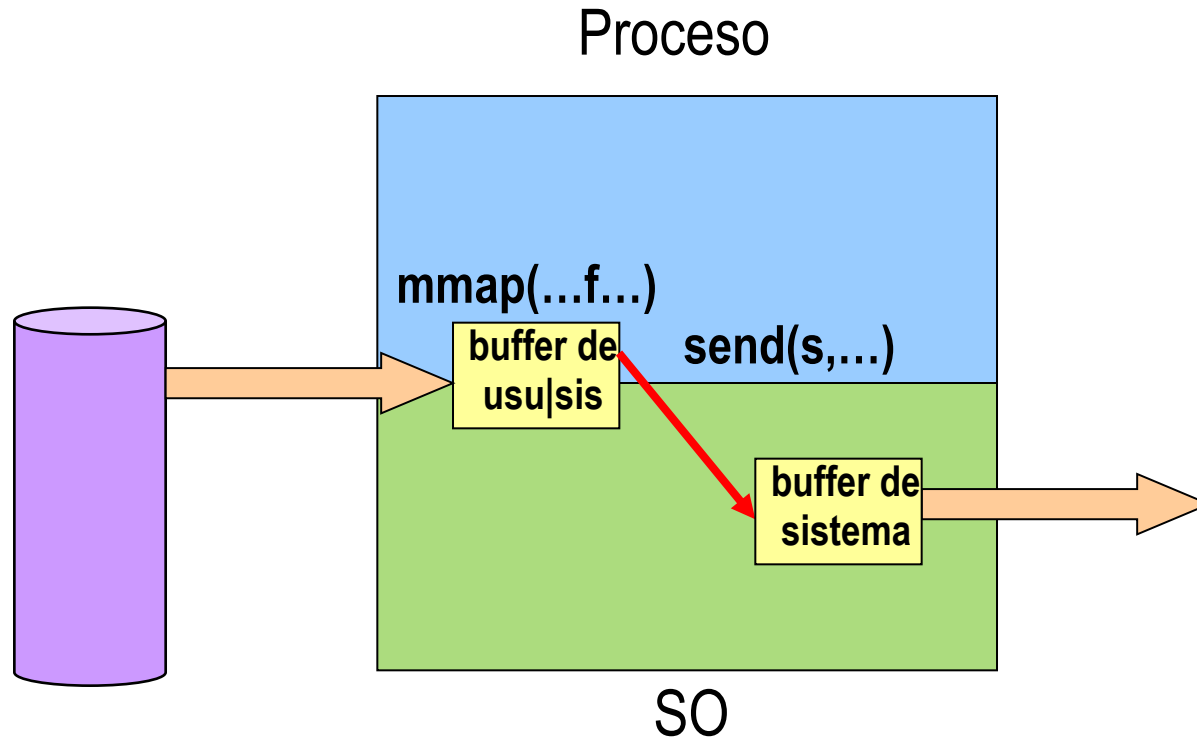
Uso de *MSG_MORE* en *send*

Envío convencional de fichero



sobrecarga de 2 copias

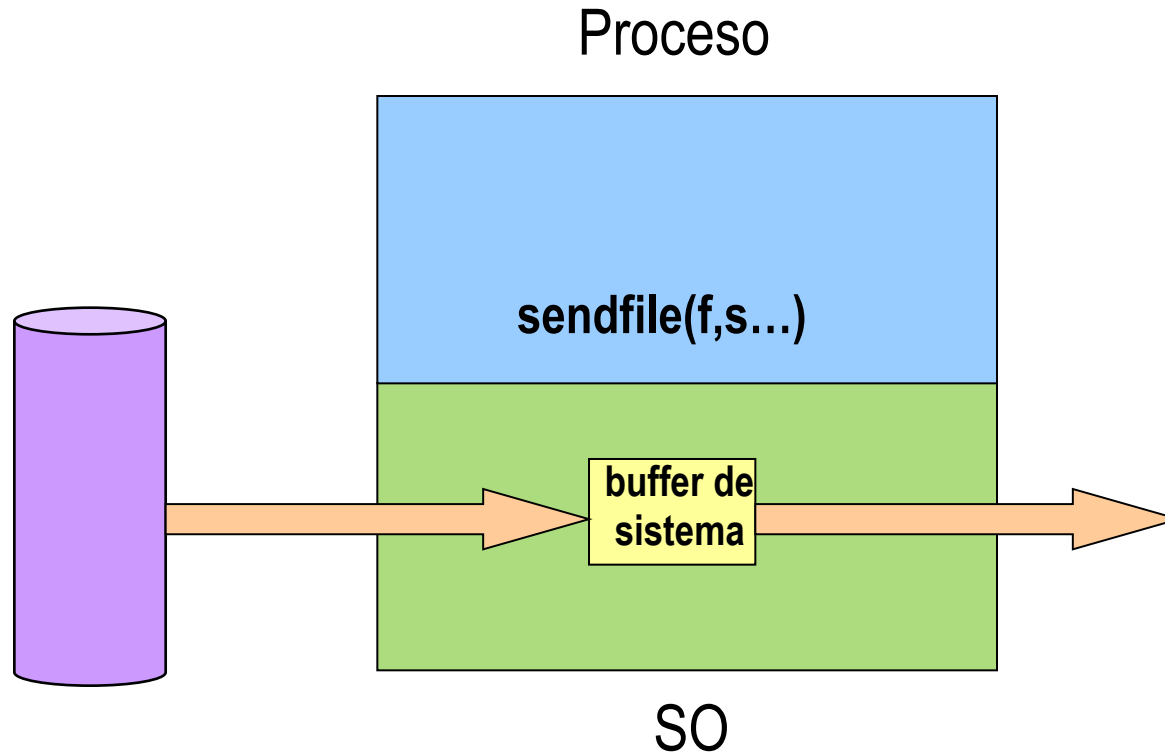
Envío con proyección de fichero



sobrecarga de 1 copia

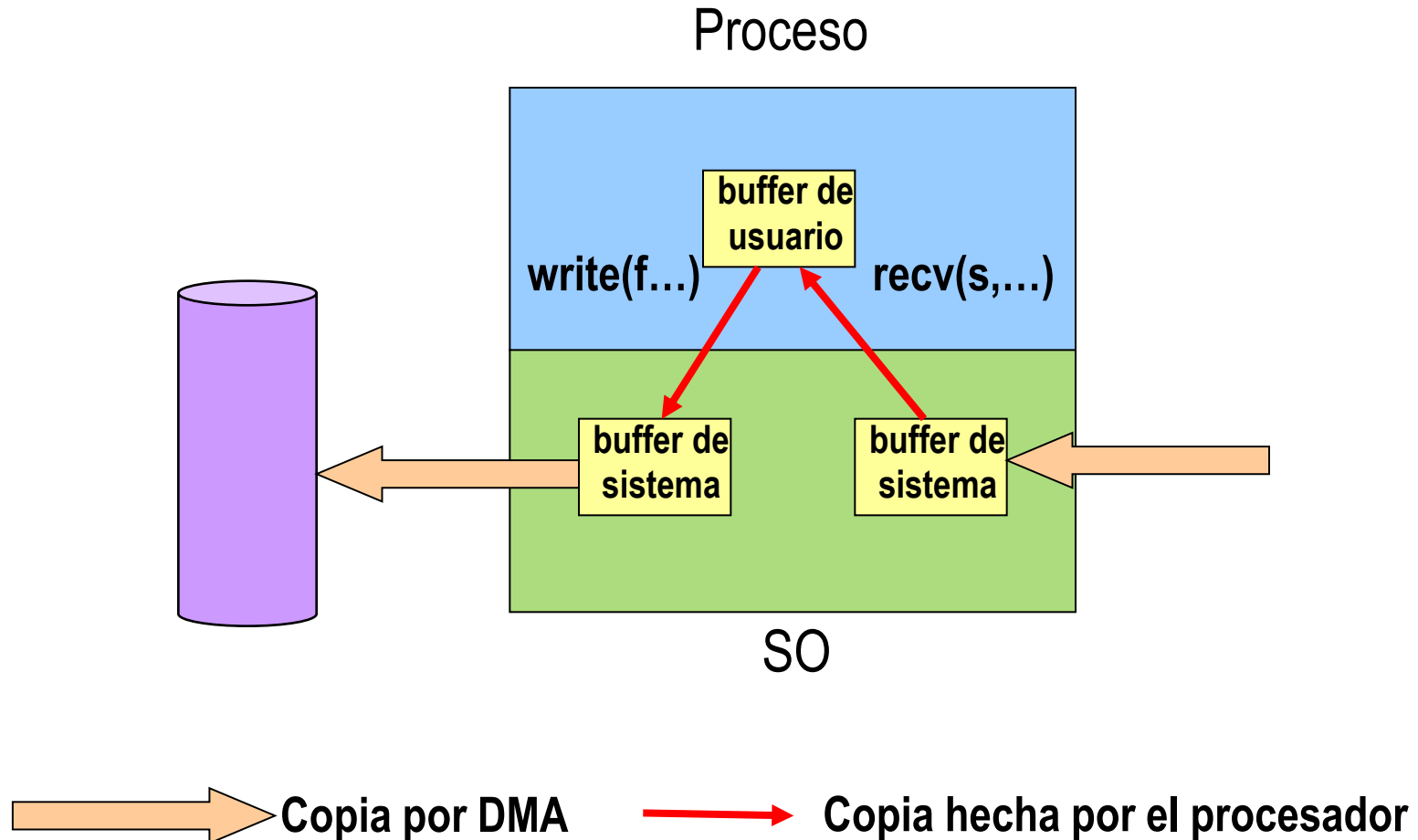
Envío zero-copy de fichero

se requiere en la práctica



sin copias adicionales

Recepción convencional de fichero



Recepción con proyección de fichero

se requiere en la práctica

